Movie Recommender System

A from scratch comparison of recommendation methods on MovieLens

Recommender Systems, Individual Project

Prof. Marc Torrens, ESADE

Jan Erik Sternberg

Goal and approach

What the assignment asks for, and the choice made here

- Build a working movie recommender prototype and evaluate it properly
- Grading: 50% technical implementation, 30% evaluation, 20% user experience
- Every model is implemented from scratch

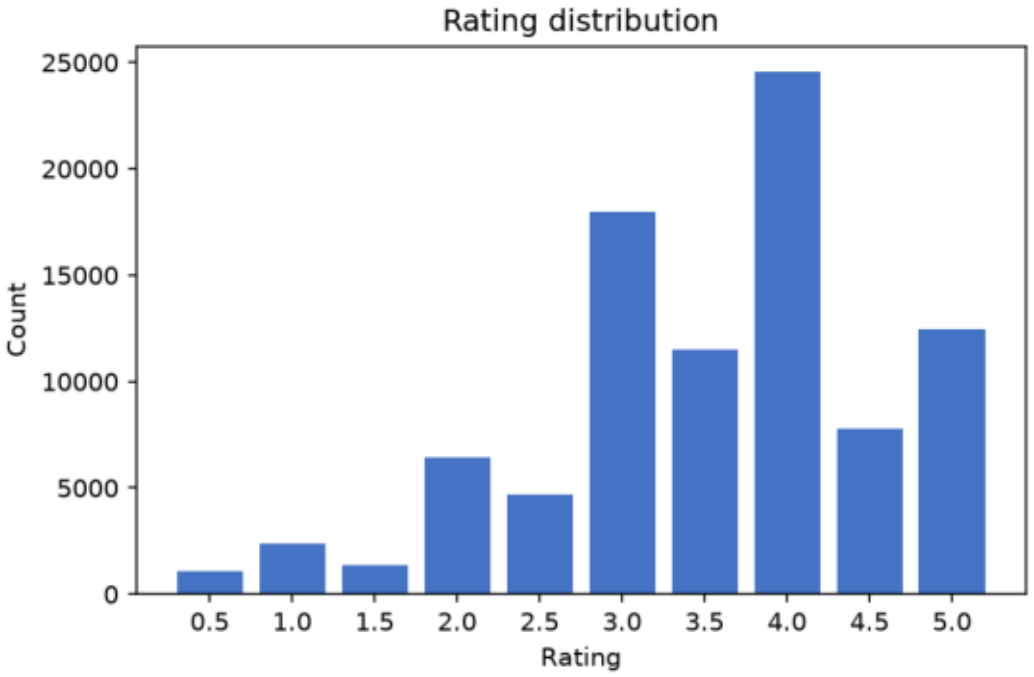
  no recommender library calls: similarity, SGD updates, and ranking are hand written
- Dataset: MovieLens latest small (610 users, 9,742 movies, 100,836 ratings)

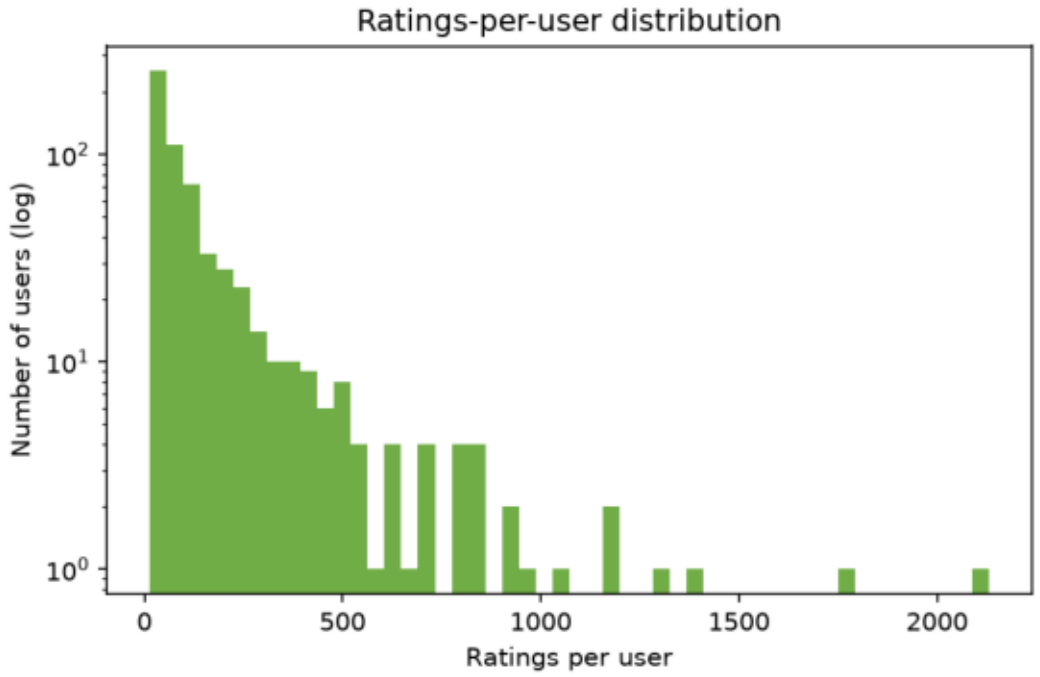
  the full 32M rating dump was kept locally but not used, it is too heavy for the non vectorized code here
- Deliverable: a Streamlit prototype plus this slide deck

  targets the user experience component directly

Dataset at a glance

Rating distribution and activity skew





Ratings skew positive (most ratings are 3.5 to 5.0), and a small number of very active users account for a large share of all ratings, a typical long tail pattern.

Evaluation protocol

One shared protocol for every model

- Per user temporal holdout split
  - each user's most recent 20% of ratings go to test, the rest to train
- Relevance threshold: a test rating of 4.0 or higher counts as relevant
  - used for ranking metrics
- Ranking metrics: Precision, Recall, NDCG, MAP, MRR, Hit Rate, all at K=10
- Beyond accuracy: catalog coverage, novelty, intra list diversity, popularity bias
- Rating prediction: RMSE and MAE on held out ratings (matrix factorization only)

Seven models, one interface

Every model implements fit() and recommend()

Non personalized

Most Popular, Highest Average Rating, Random

Content based

TF-IDF over genres and tags, centered rating weighted user profile, cosine scoring

Collaborative filtering

Item item and user user neighborhood CF with significance weighted (shrunk) cosine similarity

Matrix factorization

FunkSVD: r-hat = mu + b_u + b_i + p_u . q_i, trained with hand rolled SGD

Technical challenge: collaborative filtering artifacts

Two sparse-data bugs found by inspecting real recommendations, not just metrics

- Bug 1: items or users sharing a single rater get cosine similarity 1.0

  fixed with significance weighted shrinkage: sim *= n_common / (n_common + alpha)

- Bug 2: every score collapsed to exactly 5.0

  a fixed global top-k of neighbors rarely overlapped a user's own rated items with ~9.7K items, so the weighted average reduced to one rating

- Fix: item item CF selects its top-k neighbors dynamically per recommendation,

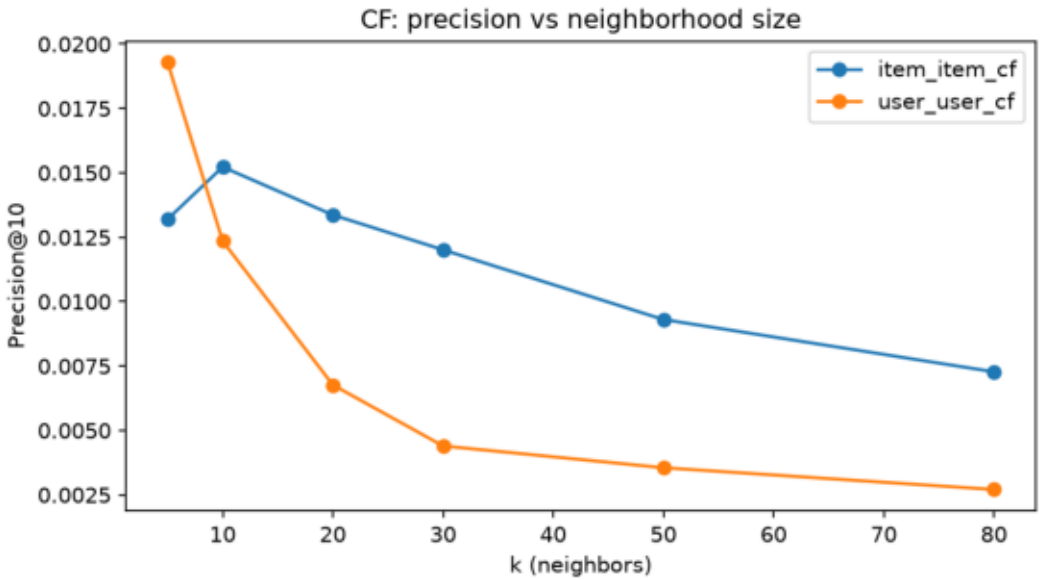
  restricted to the items the target user actually rated, instead of a global top-k computed once at fit time

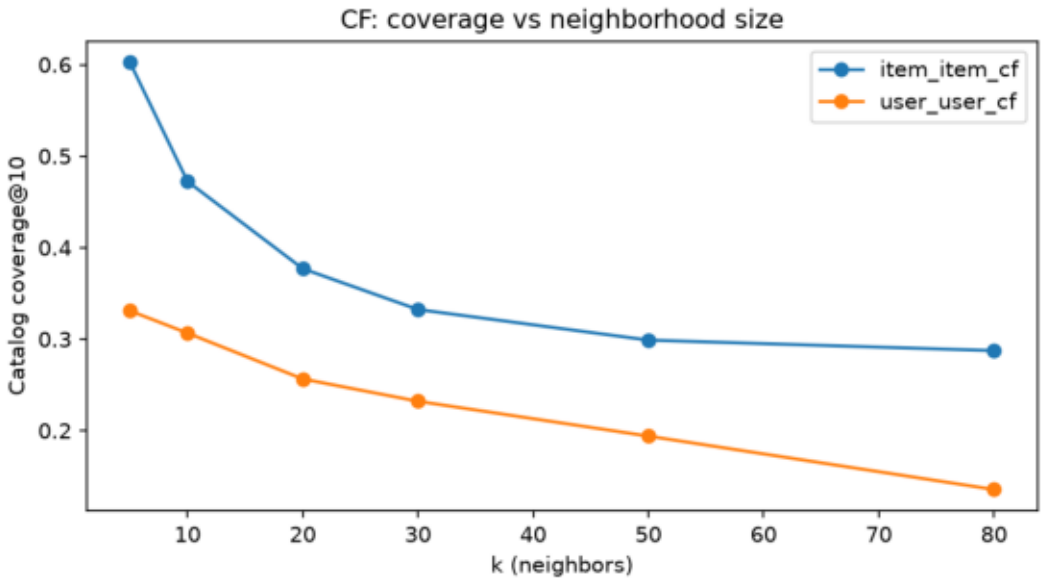
- User user CF keeps a global top-k

  with only about 610 users this failure mode does not apply

Tuning: neighborhood size k

Precision and catalog coverage trade off as k grows

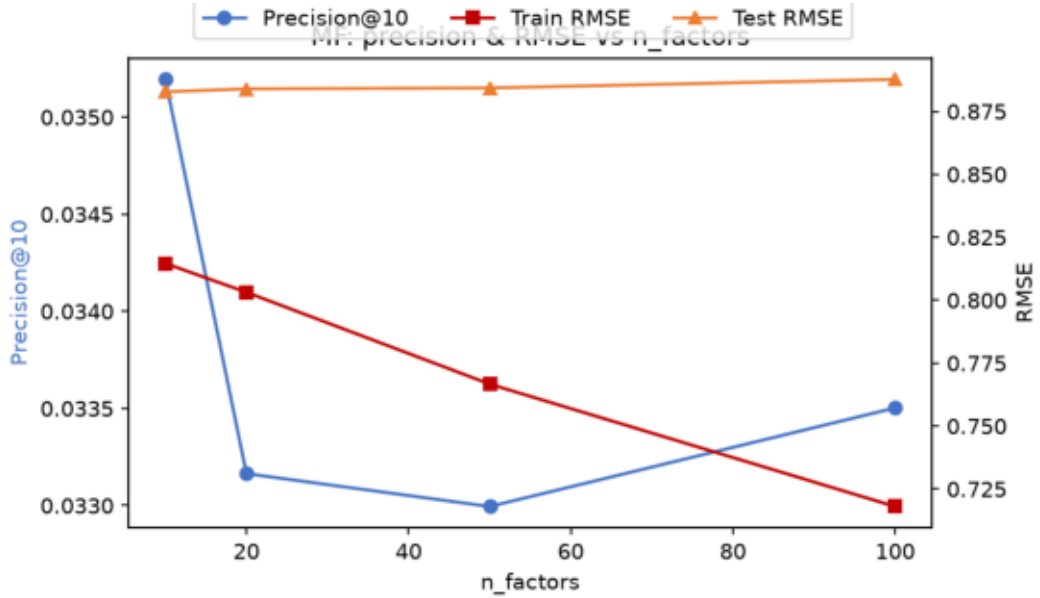


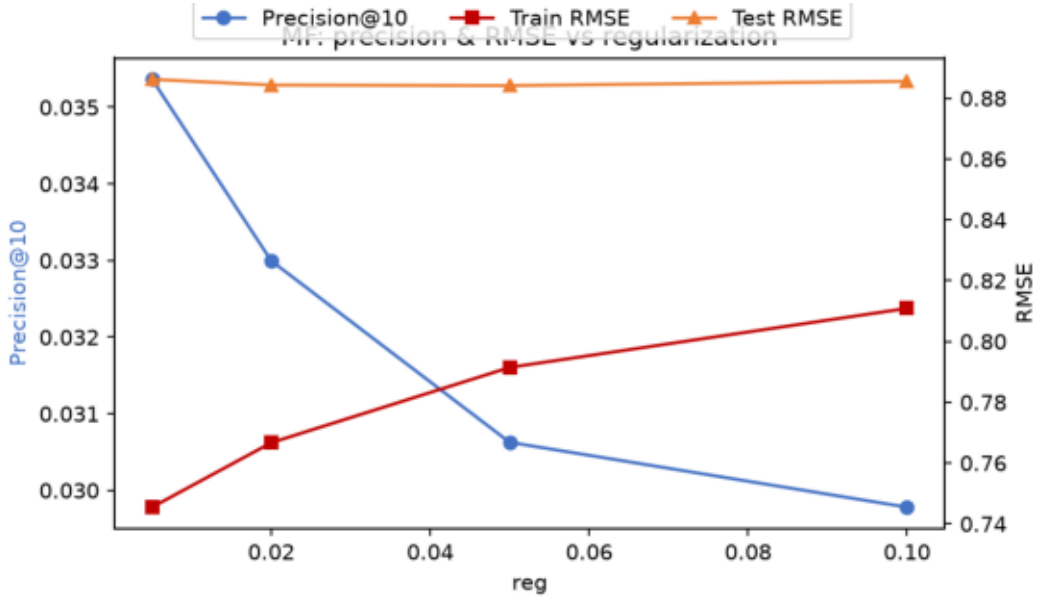


Smaller neighborhoods give both better precision and higher coverage:
user user peaks at k=5, item item at k=10. Larger k narrows recommendations toward the same popular items.

Matrix factorization: finding overfitting

Train RMSE alone hides it, held out test RMSE reveals it

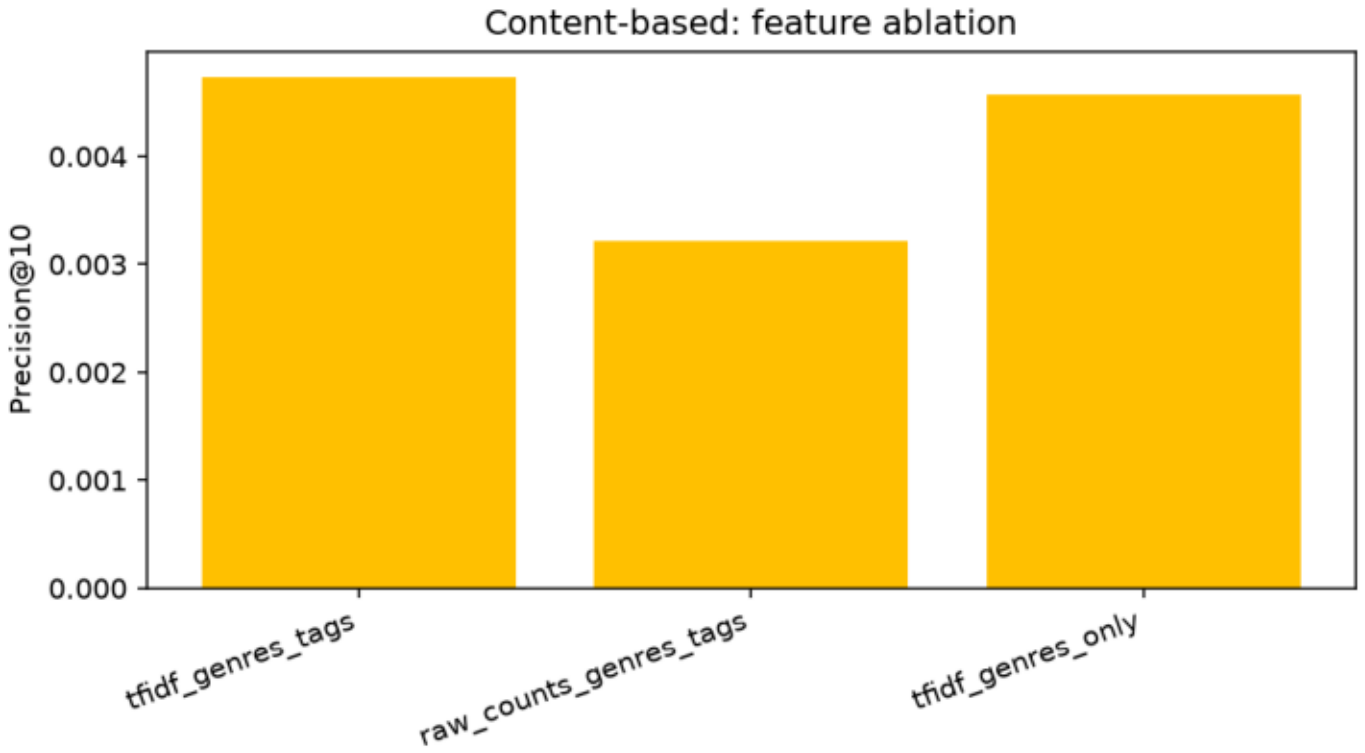




Train RMSE keeps falling with more latent factors or less regularization (0.81 to 0.72),
but test RMSE stays flat near 0.88, and ranking quality peaks at small, well regularized settings.

Content based: feature ablation

TF-IDF versus raw counts, genres only versus genres and tags



- TF-IDF beats raw counts

  0.0047 vs 0.0032 precision at 10

- Genres only is close to genres and tags

  0.0046 vs 0.0047

- but with much higher coverage

  0.91 vs 0.85

- Tags add little signal here while narrowing variety

Overall results

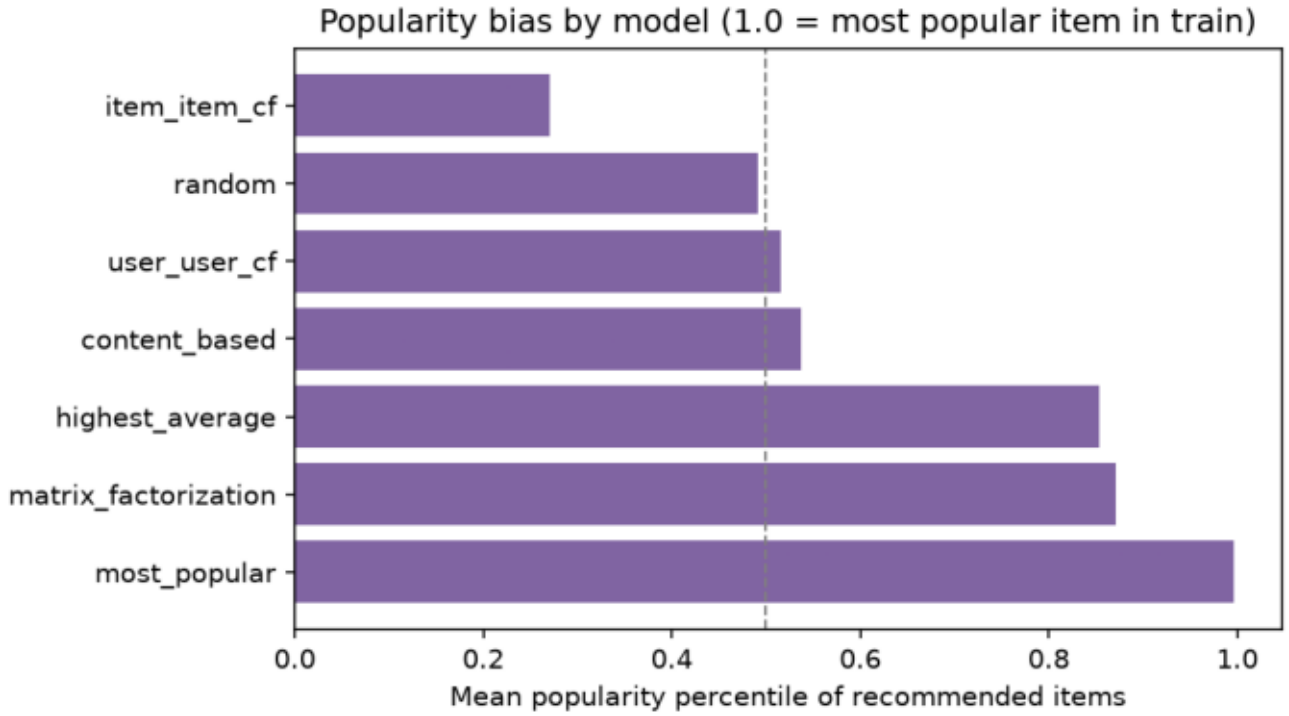
Precision, recall, and NDCG at K=10, full test set

Model	Precision@10	Recall@10	NDCG@10	Coverage@10
Most Popular	0.058	0.050	0.076	0.026
Highest Average	0.023	0.018	0.031	0.011
Matrix Factorization	0.031	0.026	0.038	0.060
Item Item CF	0.013	0.004	0.013	0.377
Content Based	0.005	0.006	0.007	0.851
User User CF	0.004	0.005	0.005	0.232
Random	0.004	0.004	0.004	0.792

Popularity baselines win on raw accuracy, expected in offline evaluation since future interactions skew toward broadly liked items. FunkSVD is the strongest personalized model.

Beyond accuracy: popularity bias

Mean popularity percentile of recommended items, 1.0 is the most popular item in train



- Most Popular: 0.997, Matrix Factorization: 0.871

  lean heavily toward blockbusters

- Content based, user user CF, and random sit near 0.5

  roughly neutral

- Item item CF: 0.270, the only model with negative bias

  it systematically surfaces below average popularity items

- This is invisible to precision alone, but matters for a real product

Prototype: Streamlit app

Targets the user experience grading component

- Pick any user and see their liked movies alongside live recommendations
- Switch between all seven models and compare scores side by side
- Adjustable recommendation list length
- Deployed and publicly reachable, no setup required to try it

Live demo: https://recommendersystemmovies.streamlit.app/

Limitations and final remarks

- Content based profiles degrade for extreme power users
  - thousands of ratings across nearly every genre wash out the taste signal in the centered weighted sum
- Offline accuracy metrics favor popularity
  - they do not capture novelty or discovery value, which is why beyond accuracy metrics matter
- Matrix factorization needs careful regularization
  - more capacity alone does not generalize better, confirmed by held out test RMSE
- Overall: building every model from scratch surfaced concrete, debuggable failure modes
  - that using a library would likely have hidden